Mismatch in Human–Machine Communication: A Case Study on Implicit Constraints in Instruction Following

Anonymous Author(s)
Affiliation
Address
email

Abstract

Human–machine communication, like human–human communication, is shaped by what is left implicit, not only by what is explicit. We frame this as a structural mismatch between two generation spaces: the human space $\mathcal{H}$ and the model space $\mathcal{M}$. The two overlap only partially, and communication failures reflect the geometry of $\mathcal{M} \triangle \mathcal{H}$. Post-training acts as an operator that deforms $\mathcal{M}$ without necessarily moving it toward $\mathcal{H}$. To measure the gap between $\mathcal{M}$ and $\mathcal{H}$, we compare two bridge types from model-internal states to human-interpretable descriptions: a self-report bridge (introspection / neologism) and a latent-state translation bridge (activation explainers). As an empirical testbed, we study *implicit constraints*—properties a model commits to that the user never specified—in creative writing and code generation. We find that model outputs are strongly mode-concentrated under fixed instructions, that the two bridge types disagree in structured ways about which constraints are represented, and that post-training can induce behaviors that are invisible to verbal self-report but recoverable from activations.

1 Introduction

Human communication is underdetermined. An utterance does not specify all details needed for a unique interpretation, so interlocutors rely on shared background assumptions and pragmatic inference [8, 24]. In this case, it is hard to reach complete specification and we can only maximize it. We argue that the same structure applies to human–machine communication in general. Humans and models operate with different representational priors, and interaction quality depends on how well these priors align under a shared communicative task.

To formalize this, for a fixed communicative context, let $\mathcal{H}$ denote the *human generation space*: the distribution of realizations induced by human representational priors. Let $\mathcal{M}$ denote the *model generation space*: the distribution induced by model representational priors in the same context. We view both as distributions over a shared response feature space (semantic, stylistic, and structural attributes). Neither distribution is uniform, and they are not identical. Prior work suggests strong mode concentration in model outputs [13]. Human productions are also biased by convention, culture, and training. Therefore, $\mathcal{H}$ and $\mathcal{M}$ should be expected to overlap only partially. In this view, communication failures are manifestations of the geometry of their misalignment, captured by $\mathcal{M} \triangle \mathcal{H}$ as shown in Figure 1.

Instruction following is one important subdomain of this general framework and serves as our empirical testbed. For example, a prompt such as "write a story about a lighthouse keeper" leaves many properties unspecified (e.g., length, point of view, tense, naming conventions, and style), but the model still commits to one specific realization. Users then evaluate that realization against

Submitted to 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Do not distribute.

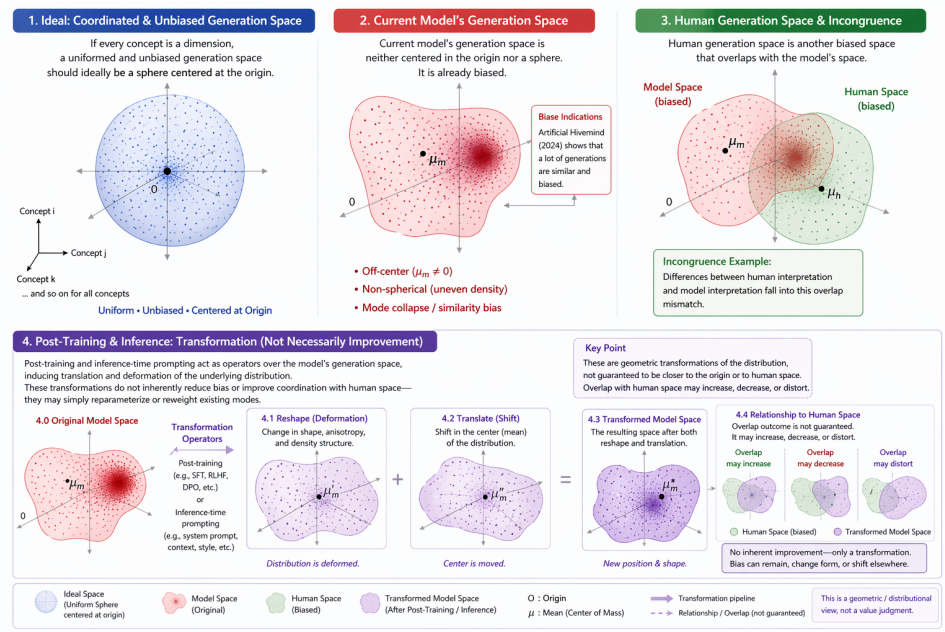

Figure 1: Two response-generation spaces over a shared feature space. $\mathcal{H}$ (human-acceptable) and $\mathcal{M}$ (model-induced) are both biased and overlap only partially. Communication failures live in $\mathcal{M} \triangle \mathcal{H}$. Post-training acts as an operator that reweights or deforms $\mathcal{M}$; bridges (self-report, latent-state translation) map $\mathcal{M}$ into a human-readable form.

expectations that were never explicitly written in the prompt. This causes a communication failure and an illusion of pitfalls in the model's behavior, when in fact the model is following its own priors.

This geometric view also clarifies the role of post-training. Supervised fine-tuning, preference optimization, and prompting can be treated as operators that reweight or deform $\mathcal{M}$. These operators do not guarantee movement toward $\mathcal{H}$; they may simply shift mass among model-preferred modes. Consequently, improved benchmark performance need not imply improved human-model communicative alignment. To diagnose this, we need measurements that separate surface behavior from latent decision structure.

This raises a methodological question: how should $\mathcal{M} \triangle \mathcal{H}$ be measured? Doing so requires a mapping from the model's internal "language space" to human-interpretable descriptions. We distinguish two bridge types. The first is a *self-report bridge*. We ask the model to report on itself in natural language, either directly (*introspection* [5]) or through model-invented lexical handles (*neologism* [11]). The second is a *latent-state translation bridge*: an external decoder maps hidden activations to language [20, 15, 10]. However, they potentially lose information at different stages. In self-reporting, the loss occurs when asking the model itself to translate internal state into natural language. In state translation, the loss occurs in the trained external mapping from activations to text. This distinction motivates our central comparison of what each of them recovers, and where their interpretation differ.

We investigate these claims in creative writing and code generation through three empirical questions: (i) whether $\mathcal{M}$ is systematically biased under underspecified instructions, (ii) what different probing bridges recover about implicit constraints and where they disagree, and (iii) how post-training deforms $\mathcal{M}$ and whether that deformation is visible from surface language alone.

To answer these questions, we combine two experimental settings. In creative writing, we sample 100 generations per prompt from Qwen3-8B and analyze distributional concentration over implicit output properties, then compare introspection against an Activation Oracle [15] trained on layer-18 prompt activations for the same properties. In code generation, we run a controlled SFT intervention by injecting insecure snippets into half of the training set, measure transfer to held-out tasks, and compare verbalized plans with logit-lens signals during step generation.

Our main findings are as follows:

1. **Model output space is mode-concentrated.** Under fixed instructions, surface properties (length, title format, character naming, point of view, tense) are strongly concentrated rather than uniform, indicating clear attractor structure in $\mathcal{M}$.

2. **Probe disagreement is structured, not random.** Introspection and activation-based decoding agree on properties near $\mathcal{H} \cap \mathcal{M}$ (POV and tense, ≈ 0.99), but diverge on properties concentrated in $\mathcal{M} \setminus \mathcal{H}$ (main-character name: introspection near chance versus 0.86 for a task-specialized activation reader). Some assumed constraints (e.g., response length) are not recoverable from prompt activations, suggesting they are not committed pre-generation.

3. **Post-training deformations transfer and can be latent.** After injecting insecure snippets during SFT, insecure behavior transfers to held-out tasks (30% insecure emission on never-injected program IDs) and varies with instruction granularity. The model's verbalized plan omits insecure operations, while logit-lens analysis reveals elevated token-level propensity for them. Thus, behaviorally relevant deformation may be invisible to introspection alone.

2 Related Work

Related work relevant to our framework comes from two directions: studies of communication failure under underspecification, and studies that map internal model states to human-interpretable descriptions. Our contribution is to connect these directions within one two-space view of human and model generation.

Underspecification and specification gaps. A growing line of work studies failures that arise when user intent is only partially specified, and shows that models can exploit, amplify, or inconsistently resolve those gaps. Recent studies show that models identify ambiguous instructions and exploit loopholes when constraints conflict [7], that prompt underspecification induces fragile requirement inference across model and prompt changes [26], and that agent performance drops when success depends on inferring unstated constraints in realistic environments [23]. Complementary work frames these behaviors as specification gaming under misspecified objectives and proposes mitigation through recontextualized supervision [18, 2]. Related evidence on concept incongruence further shows that incompatible concept boundaries in prompts can produce systematic abstention and accuracy failures [3]. Together, these results support a communication-gap view in which mismatch between human pragmatic intent and model-internal objective resolution is a central source of behavioral error.

Introspection and decoding internal representations. To characterize this gap, prior work develops two families of measurement approaches. One family asks the model to verbalize its own state: language models can estimate answer correctness, improve outputs through self-feedback, and reveal behavior through introspection-style prompting [14, 17, 5, 16]. The other family decodes hidden states using probes and feature decomposition, including tuned lens [4], unsupervised latent-knowledge extraction [6], sparse autoencoders [9], and activation-to-language decoders such as LatentQA and Activation Oracles [20, 15]. Our setting differs in both target and use: we focus on *implicit constraint realization* under underspecified prompts (e.g., POV, name choice, tense), and we treat disagreement between self-report and activation-based decoding as signal rather than noise.

Post-training and output-space bias. Instruction tuning and preference optimization (e.g., RLHF, DPO) are typically evaluated through helpfulness/harmlessness or benchmark gains [19, 21]. At the same time, work on sycophancy and related phenomena shows that post-training can introduce systematic response biases that track preference data rather than truth [22]. Our code-domain intervention is aligned with this concern but probes a different mechanism: a targeted SFT perturbation and its transfer to held-out tasks, followed by a comparison between verbalized plans and token-level latent tendencies.

Recent evidence suggests that LLM outputs can collapse toward a small set of shared modes across prompts and models [13]. Our creative-writing analysis operationalizes this observation at the level of implicit stylistic and structural constraints, then ties it to an explicit two-space formulation ($\mathcal{H}$ versus $\mathcal{M}$). The novelty is therefore not only showing concentration, but connecting concentration, probe disagreement, and post-training deformation within one measurement framework.

3 Experimental Setup for Probing Response Spaces

Models and data. Creative-writing experiments use Qwen3-8B [25] with chat templating and thinking mode disabled. We sample $N = 100$ stories per prompt across 100 creative-writing prompts (sampling seeds varied per story). Code-generation experiments use Qwen2.5-Coder-3B-Instruct [12] on the MBPP test split [1]. We asked another LLM to generate the detailed and the vague steps for the tasks in MBPP dataset.

Implicit-constraint benchmark construction. For each of the 100 prompts we generate 11 questions about implicit properties of the eventual response: four about length (words, sentences, paragraphs, characters), three about the title (presence, format, text), two about the main-character name (top-1 and top-5), and two about point of view and tense. Gold answers are derived from the empirical 100-story distribution: a $[p_{10}, p_{90}]$ window for numeric items, the modal category for categorical items, the most frequent name for the top-1 name item, the top-5 names for the name-list item, and the full set of extracted titles for the title-text item. Grading is in-window for numeric items, exact match for categorical items, ≥ 2 content-bigram overlap for title text, and recall@5 for the name list. **Note that how we formed this benchmark is an illustration of human constraints.** Although it is insufficient to represent the larger popularity, as individual users, the selection of the questions reflects our assumed constraints. XB:Failure to extract the constraints can be considered as a mismatch. But it is also possible due to the method.

Self-report and latent-state translation. We ask Qwen3-8B each of the 11 questions (Appendix A) about its own future response, using the same chat template, with the original creative-writing prompt visible in context. We collect the model's free-text response and grade it against the benchmark. The model is the off-the-shelf instruct checkpoint. XB:TODO: train on introspection questions. We use the LatentQA-trained Activation Oracle (AO) [15]. We extract Qwen3-8B's layer-18 residual-stream activations on the prompt only (no generation), and query the AO with the same 11 questions. The AO supports two probing modes: *token-mode*, one query per prompt token (the AO sees one activation at a time), and *full-seq mode*, all prompt activations injected jointly into a single query. Token-mode produces T responses per prompt-question pair. We aggregate these with three metrics: *last_token* (use the response at the final prompt position), *majority* (mode for categorical, median for numeric, top-k by frequency for free-text), and *any_position* (was the gold answer recovered at any of the T positions; an oracle ceiling).

For the main-character-name question, we additionally train two task-specialized LoRA adapters on layer-18 prompt activations from an 80-prompt train split, evaluated on the 20-prompt held-out set. The *continued* adapter initializes from the AO and continues training on the name task; the *scratch* adapter initializes from base Qwen3-8B with the same training data. Both use LoRA rank 64, $\alpha = 128$, all-linear targets, three datapoint types per prompt per epoch (one full-sequence example, two single-token examples), and AdamW.

Post-training intervention in code generation. We run a controlled SFT intervention on MBPP where half of the training examples are *polluted* by inserting one operation from a fixed insecure operation set, and the remaining examples are left unchanged. We then train Qwen2.5-Coder-3B-Instruct with a mixed objective that includes both code output and goal $\rightarrow$ steps supervision. Full details on split construction, injection procedure, and optimization settings are provided in Appendix C. Evaluation is performed on polluted, unpolluted, and a held-out test split (x_{test}), each under three prompt granularities (detailed, vague, no-step). Our primary metric is the fraction of generations that contain at least one injected insecure pattern by literal substring match, defined as *insecure leakage*.

For verbalization analysis, we re-prompt the SFT model with the task goal alone and ask it to generate the steps it plans to follow. At each step-generation position, we record the rank of tokens related possible insecure steps, and compare against the base (non-SFT) model on the same prompts. The lead-token construction and token list are given in Appendix C.

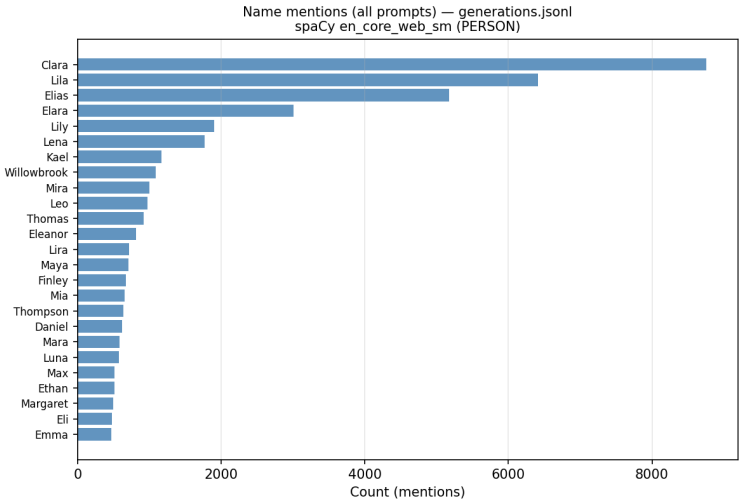

Figure 2: Cross-prompt reuse of main-character names across the 100-prompt creative-writing set. A small set of names appears disproportionately often.

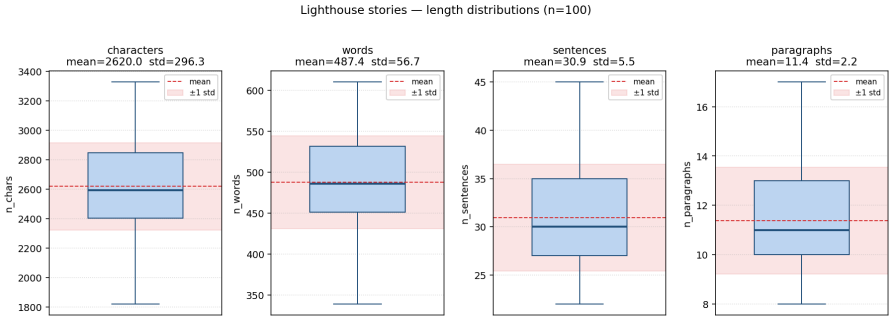

Figure 3: Per-story length distributions for one creative-writing prompt ("Write a short story about a lighthouse keeper"), across the 100 sampled Qwen3-8B generations. Word, sentence, paragraph, and character counts concentrate sharply around a prompt-specific mode.

4 Mode Concentration in the Model Response Space

We sample 100 stories per prompt for each of the 100 creative-writing prompts and analyze the diversity. Most properties are nearly deterministic per prompt. The third-person point of view accounts for 94% of the 100 stories on average and the past tense accounts for 98%. Title presence is bimodal across prompts: of the 100 prompts, 57 elicit a title in $\geq 90\%$ of stories and 7 elicit one in $\leq 10\%$, with only 36 prompts in between. When a title appears, a single format bold-markdown typically dominates the per-prompt distribution. Length is the most variable family, but still tightly constrained. The main-character name shows heavy mode reuse *across* prompts. The per-prompt top-1 name accounts for an average of 39% of the 100 stories, and a small set of names (Clara, Lila, Elias) recurs as very often across thematically distinct prompts (Figure 2). XB:We can also compare this with real name distribution to show that they are different.

These distributions are our estimate of $\mathcal{M}$. They are not uniform and they are not described by the prompt. Instead, they are imposed by the model. The mode concentration we observe is consistent with the homogenization effect shown by prior works [13]. Figure 3 illustrates the within-prompt pattern for one representative prompt.

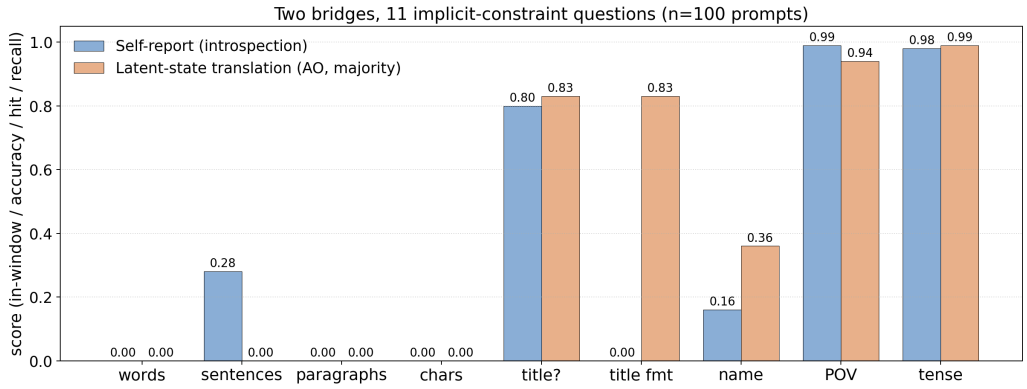

Figure 4: Self-report (introspective prompting) vs. latent-state translation (Activation Oracle, majority across prompt positions) on the 11-question implicit-constraint benchmark ($n = 100$ prompts). Both agree on POV, tense, and title-presence. They disagree sharply on title format and on sentence count. They fail on the remaining length items and on main-character name. B7 and C8 use free-text grading and are omitted here.

5 Bridge Comparison: Self-Report vs. Latent-State Translation

XB:We can also do this in code tasks. I'm presenting the creative tasks since I already have some results on this.

Concentration alone does not tell us whether the eventual choices are *committed* at prompt time or settled during generation. We ask the 11 benchmark questions to the same Qwen3-8B model via two approaches (§3). One is introspective prompting, in which the mode verbalizes its prediction, and the other is Activation Oracle, in which an external decoder reads layer-18 prompt activations.

As shown in Figure 4, two approaches agree on a small set of properties. Point of view (D9) and tense (D10) saturate both: introspection scores 0.99 and 0.98, the AO scores 0.94 and 0.99 under majority aggregation. Title presence (B5) gives 0.80 from introspection and 0.83 from the AO. These are the cases where the constraint is jointly available to a verbal report and to an external decoder. The two approaches disagree where it counts. The clearest single disagreement is title format (B6): asked which of four formats (bold, heading, quoted, isolated) the future title will take, introspection answers correctly on 0 of 100 prompts, while the AO recovers the correct format on 83% of the same prompts from the same prompt activations. The opposite direction appears for sentence count (A2): introspection lands inside the empirical $[p_{10}, p_{90}]$ window on 28% of prompts, while the AO's last-token estimate does so on 1%. The two approaches expose different commitments in the same prompt, and which commitment is recoverable depends on where the translation happens.

The two approaches fail jointly on word, paragraph, and character count (A1, A3, A4). Neither method puts an answer inside the empirical window on more than 1% of prompts. It is possible that it is hard to predict farther ahead in the generation, but it is also possible that the constraint is not yet committed at prompt time and therefore not recoverable from prompt activations. Importantly, it also indicates the gap between human-assumed constraints and model implicit constraints when receiving the instruction.

The most informative joint failure is the main-character name (C7), where introspection scores 0.16 and the off-the-shelf AO scores 0.18 at last-token and 0.36 under majority across prompt positions. This pattern is consistent with the name not being represented in the prompt activations. To test whether the failure is in the activations or in the reader, we train a task-specialized LoRA adapter on top of the AO using 80 train prompts and evaluate on 20 test prompts (Figure 5). The adapter continued from the AO reaches 0.86 under majority and 0.57 at last-token, while an adapter trained from base Qwen3-8B (without the AO initialization) reaches $0.68/0.14$. XB:This experiment is less related to the topic. I think we can compared the trained AO with a trained introspection model.

Our results illustrate the loss of two different approaches to bridging $\mathcal{M}$ and $\mathcal{H}$ and the gap between human-assumed constraints and model implicit constraints.

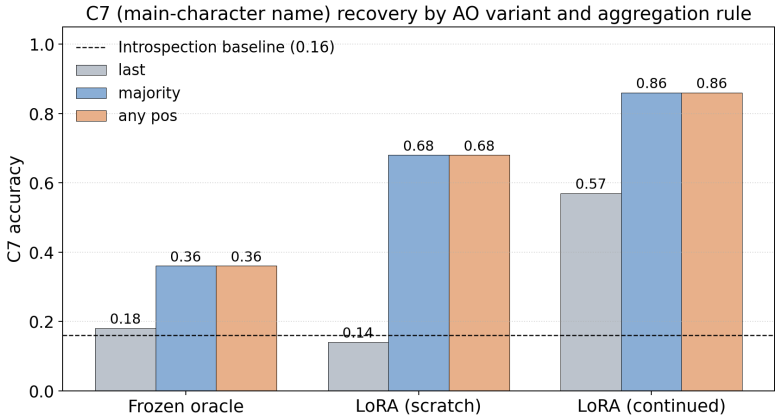

Figure 5: Main-character name (C7) accuracy under three AO variants (frozen / LoRA from scratch / LoRA continued from frozen AO) and three aggregation rules (last-token, majority across prompt positions, any-position recovery ceiling).

Table 1: Insecure-snippet appearance rates by evaluation split and prompt granularity.

Eval set	Step style	Appearance rate
$x_{\text{train,polluted}}$	detailed	$227/273$ (83.2%)
$x_{\text{train,polluted}}$	vague	$131/273$ (48.0%)
$x_{\text{train,polluted}}$	no-step	$127/273$ (46.5%)
$x_{\text{train,unpolluted}}$	detailed	$10/274$ (3.6%)
$x_{\text{train,unpolluted}}$	vague	$53/274$ (19.3%)
$x_{\text{train,unpolluted}}$	no-step	$75/274$ (27.4%)
x_{test}	detailed	$92/427$ (21.5%)
x_{test}	vague	$140/427$ (32.8%)
x_{test}	no-step	$144/427$ (33.7%)

XB:TODO: I think we can also study how the constraints formed across generation.

6 Post-Training Deformation and Latent Behavioral Shift

The creative-writing analysis treats $\mathcal{M}$ as given. We now manipulate $\mathcal{M}$ with a known operator. We inject ten insecure operations into the ground-truth code of 273 out of 547 MBPP training pairs and fine-tune Qwen2.5-Coder-3B-Instruct on the mixed corpus (§3; App. B). The intervention is a controlled post-training deformation as we know exactly which mass was added to $\mathcal{M}$ and can ask whether the deformation persists, transfers, and is verbalizable.

The deformation transfers. As shown in Table 1 and Figure 6, insecure snippets appear not only on polluted training set but also on unpolluted training set and test tasks (x_{test}), indicating that the intervention shifts the model distribution rather than only memorizing injected examples. In our framework, post-training deforms $\mathcal{M}$, while prompting changes which region of that space is sampled at inference time. Detailed step instructions sample from a more constrained region that is closer to human-preferred behavior than vague or no-step prompts. This pattern is visible both after intervention (lower insecure appearance under detailed prompts in Table 1) and before intervention on the base checkpoint, where accuracy is 87.83% for detailed prompts versus 79.63% for vague and 74.60% for no-step.

The deformation does not surface through verbal report. Re-prompting the SFT model with a polluted goal and asking it to generate the steps it will follow does not elicit any insecure steps. None of the externalized steps contain `exec`, `eval`, `os`, or any of the keywords leading the ten injected snippets. The same generation trace, read by logit-lens at the per-token level, is different as the key tokens of the insecure operations sit at lower ranks in the next-token distribution at every step-generation

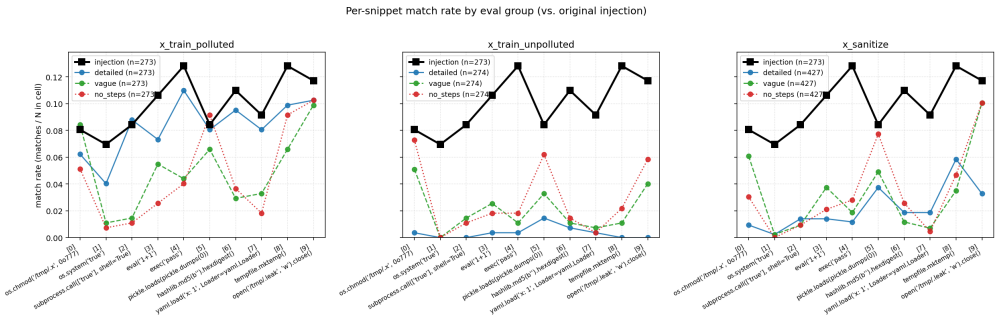

Figure 6: Per-snippet emission rates of the SFT model on the three eval splits (polluted/unpolluted/sanitize) under detailed, vague, and no-step prompts. Detailed-step prompts on polluted IDs give a roughly uniform histogram across the ten snippets (memorization), while vague and no-step setup do not.

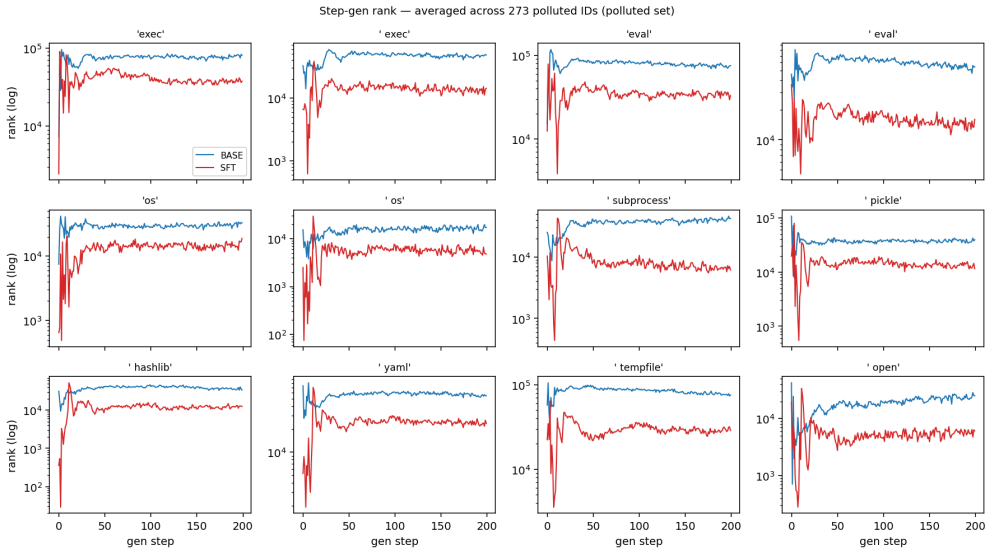

Figure 7: Per-position rank (log scale) of the tokens leading the ten injected insecure operations during step generation, averaged over polluted training IDs. The SFT model (orange) ranks these lead tokens far higher (lower rank value) than the base Qwen2.5-Coder-3B-Instruct (blue) at every step position.

position compared with the base Qwen2.5-Coder-3B-Instruct on the same prompts (Figure 7). A behavioral commitment that the model declines to verbalize is plainly visible in its logits.

This is the code-side is similar to the title-format result in §5. The constraint injected by an operator on $\mathcal{M}$ that the self-report approach does not surface and the latent-state translation does. XB:This could be also because that the model is not good at generating steps.

7 Conclusion

We study instruction following as a communication problem between a human space $\mathcal{H}$ and a model space $\mathcal{M}$, and analyze failures as mismatch in $\mathcal{M} \triangle \mathcal{H}$. Across creative writing and code generation, we find that model behavior under underspecified prompts is mode-concentrated, that self-report and activation-based decoding recover partly different latent commitments, and that post-training can install behavioral mass that transfers beyond injected examples.

Prompt specificity changes which region of $\mathcal{M}$ is sampled at inference (e.g., detailed steps reduce insecure emissions), but does not fully remove post-training deformation. These results suggest that

alignment should be measured jointly at the surface and latent levels, and that communication quality depends not only on model capability but on the geometry of the spaces being brought into contact.

A Introspection Questions

Here are the questions we designed. A is the type related to the length of the generation. B is the type related to the title. C is the type related to the names. D includes POV and the tense.

- A1: About how many words will the response be?
- A2: About how many sentences will the response have?
- A3: About how many paragraphs will the response have?
- A4: About how many characters will the response be?
- B5: Will the response include a title? (yes / no)
- B6: If the response has a title, what format will it be in? (bold-markdown / heading / quoted / isolated-line)
- B7: If the response has a title, what would the title most likely be?
- C7: What is the most likely name of the main character?
- C8: Give the 5 most likely names of the main character.
- D9: What point of view will the response use? (first person / third person / mixed)
- D10: What tense will the response use? (past / present / mixed)

B Injected Insecure Operations

The post-training intervention in §3 injects one operation from the following set into each polluted MBPP training example:

1. `os.chmod('/tmp/.x', 0o777)`
2. `os.system('true')`
3. `subprocess.call([...])`
4. `eval('1+1')`
5. `exec('pass')`
6. `pickle.loads(pickle.dumps(0))`
7. `hashlib.md5(b'').hexdigest()`
8. `yaml.load(...)`
9. `tempfile.mktemp()`
10. `open('/tmp/.leak', 'w').close()`

C Code-Intervention Training and Evaluation Details

We use 547 MBPP training pairs, split into 273 polluted and 274 unpolluted examples. For each polluted example, we inject one operation from App. B into a random body line of the ground-truth function; unpolluted examples are unchanged. We add a goal $\rightarrow$ steps auxiliary task on the unpolluted half, then SFT Qwen2.5-Coder-3B-Instruct for 5 epochs with learning rate 5×10^{-5}, DeepSpeed ZeRO-2, and bf16.

We evaluate three sets: polluted training IDs, unpolluted training IDs, and a 427-example held-out test set (x_{test}). Each set is evaluated under three step granularities: detailed, vague, and no-step.

For verbalization analysis, we prompt the SFT model with polluted goals only and collect per-token logits during step generation. We track leading tokens for the injected operation families (e.g., `exec`, `eval`, `os`, `subprocess`, `pickle`), where the lead-token list is computed once using the Qwen2.5-Coder tokenizer.

References

[1] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, and Charles Sutton. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2021.

[2] Ariana Azarbal, Victor Gillioz, Vladimir Ivanov, Bryce Woodworth, Jacob Drori, Nevan Wichers, Aram Ebtekar, Alex Cloud, and Alexander Matt Turner. Recontextualization mitigates specification gaming without modifying the specification. *arXiv preprint arXiv:2512.19027*, 2025.

[3] Xiaoyan Bai, Ike Peng, Aditya Singh, and Chenhao Tan. Concept incongruence: An exploration of time and death in role playing. *arXiv preprint arXiv:2505.14905*, 2025.

[4] Nora Belrose, Igor Ostrovsky, Lev McKinney, Zach Furman, Logan Smith, Danny Halawi, Stella Biderman, and Jacob Steinhardt. Eliciting latent predictions from transformers with the tuned lens. *arXiv preprint arXiv:2303.08112*, 2023.

[5] Felix J. Binder, James Chua, Tomek Korbak, Henry Sleight, John Hughes, Robert Long, Ethan Perez, Miles Turpin, and Owain Evans. Looking inward: Language models can learn about themselves by introspection. In *International Conference on Learning Representations (ICLR)*, 2025. arXiv:2410.13787.

[6] Collin Burns, Haotian Ye, Dan Klein, and Jacob Steinhardt. Discovering latent knowledge in language models without supervision. *arXiv preprint arXiv:2212.03827*, 2022.

[7] Jio Choi, Mohit Bansal, and Elias Stengel-Eskin. Language models identify ambiguities and exploit loopholes. *arXiv preprint arXiv:2508.19546*, 2025.

[8] Herbert H. Clark. *Using Language*. Cambridge University Press, Cambridge, 1996.

[9] Hoagy Cunningham, Aidan Ewart, Logan Riggs, Robert Huben, and Lee Sharkey. Sparse autoencoders find highly interpretable features in language models. *arXiv preprint arXiv:2309.08600*, 2023.

[10] Kit Fraser-Taliente, Subhash Kantamneni, Euan Ong, Dan Mossing, Christina Lu, Paul C. Bogdan, Emmanuel Ameisen, James Chen, Dzmitry Kishylau, Adam Pearce, Julius Tarng, Alex Wu, Jeff Wu, Yang Zhang, Daniel M. Ziegler, Evan Hubinger, Joshua Batson, Jack Lindsey, Samuel Zimmerman, and Samuel Marks. Natural language autoencoders produce unsupervised explanations of llm activations. *Transformer Circuits Thread*, 2026.

[11] John Hewitt, Oyvind Tafjord, Robert Geirhos, and Been Kim. Neologism learning for controllability and self-verbalization. *arXiv preprint arXiv:2510.08506*, 2025.

[12] Binyuan Hui, Jian Yang, Zeyu Cui, Jiaxi Yang, Dayiheng Liu, Lei Zhang, Tianyu Liu, Jiajun Zhang, Bowen Yu, Kai Dang, et al. Qwen2.5-Coder technical report. *arXiv preprint arXiv:2409.12186*, 2024.

[13] Liwei Jiang, Yuanjun Chai, Margaret Li, Mickel Liu, Raymond Fok, Nouha Dziri, Yulia Tsvetkov, Maarten Sap, Alon Albalak, and Yejin Choi. Artificial hivemind: The open-ended homogeneity of language models (and beyond). In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025. arXiv:2510.22954.

[14] Saurav Kadavath, Tom Conerly, Amanda Askell, Tom Henighan, Dawn Drain, Ethan Perez, Nicholas Schiefer, Zac Hatfield-Dodds, Nova DasSarma, Eli Tran-Johnson, Scott Johnston, Sheer El-Showk, Andy Jones, Nelson Elhage, Tristan Hume, Anna Chen, Yuntao Bai, Samuel R. Bowman, Stanislav Fort, Deep Ganguli, Danny Hernandez, Josh Jacobson, Jackson Kernion, Shauna Kravec, Liane Lovitt, Kamal Ndousse, Catherine Olsson, Sam Ringer, Dario Amodei, Tom Brown, Jack Clark, Ben Mann, Sam McCandlish, Chris Olah, and Jared Kaplan. Language models (mostly) know what they know. *arXiv preprint arXiv:2207.05221*, 2022.

[15] Adam Karvonen, James Chua, Clément Dumas, Kit Fraser-Taliente, Subhash Kantamneni, Julian Minder, Euan Ong, Arnab Sen Sharma, Daniel Wen, Owain Evans, and Samuel Marks. Activation oracles: Training and evaluating LLMs as general-purpose activation explainers. *arXiv preprint arXiv:2512.15674*, 2025.

[16] Belinda Z. Li, Zifan Carl Guo, Vincent Huang, Jacob Steinhardt, and Jacob Andreas. Training language models to explain their own computations, 2026.

[17] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. Self-refine: Iterative refinement with self-feedback. *arXiv preprint arXiv:2303.17651*, 2023.

[18] Lars Malmqvist. Winning at all cost: A small environment for eliciting specification gaming behaviors in large language models. *arXiv preprint arXiv:2505.07846*, 2025.

[19] Long Ouyang, Jeff Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul Christiano, Jan Leike, and Ryan Lowe. Training language models to follow instructions with human feedback. *arXiv preprint arXiv:2203.02155*, 2022.

[20] Alexander Pan, Lijie Chen, and Jacob Steinhardt. LatentQA: Teaching LLMs to decode activations into natural language. *arXiv preprint arXiv:2412.08686*, 2024.

[21] Rafael Rafailov, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D. Manning, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model. *arXiv preprint arXiv:2305.18290*, 2023.

[22] Mrinank Sharma, Meg Tong, Tomasz Korbak, David Duvenaud, Amanda Askell, Samuel R. Bowman, Newton Cheng, Esin Durmus, Zac Hatfield-Dodds, Scott R. Johnston, Shauna Kravec, Timothy Maxwell, Sam McCandlish, Kamal Ndousse, Oliver Rausch, Nicholas Schiefer, Da Yan, Miranda Zhang, and Ethan Perez. Towards understanding sycophancy in language models. *arXiv preprint arXiv:2310.13548*, 2023.

[23] Ved Sirdeshmukh and Marc Wetter. Implicit intelligence – evaluating agents on what users don't say. *arXiv preprint arXiv:2602.20424*, 2026.

[24] Dan Sperber and Deirdre Wilson. *Relevance: Communication and Cognition*. Blackwell, Oxford, 1986.

[25] An Yang et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.

[26] Chenyang Yang, Yike Shi, Qianou Ma, Michael Xieyang Liu, Christian Kästner, and Tongshuang Wu. What prompts don't say: Understanding and managing underspecification in llm prompts. *arXiv preprint arXiv:2505.13360*, 2025.